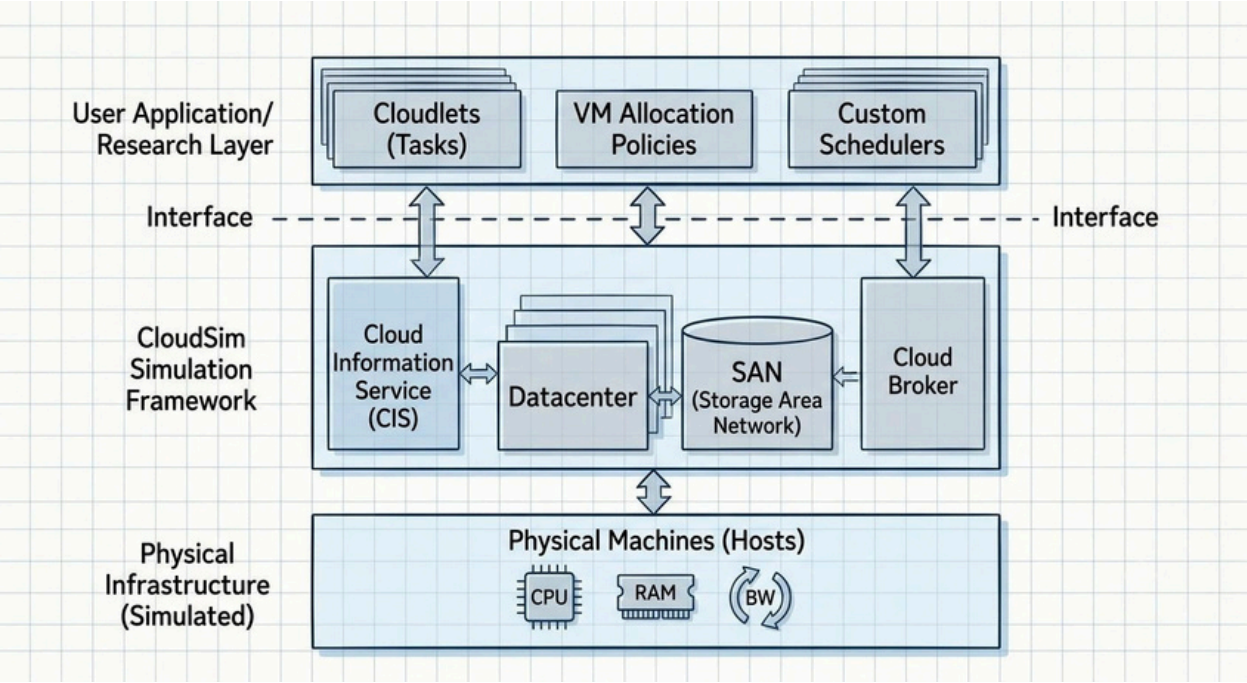
Practical 1

Aim : Sketch out and Analyze the Architecture of CloudSim and identify different entities to understand Stucture of CloudSim.

Objective : To structurally map, sketch, and analyze the component-based architecture of the CloudSim simulation framework; to identify and isolate core simulation entities; and to trace how discrete events coordinate cloud resource provisioning.

Architecture :



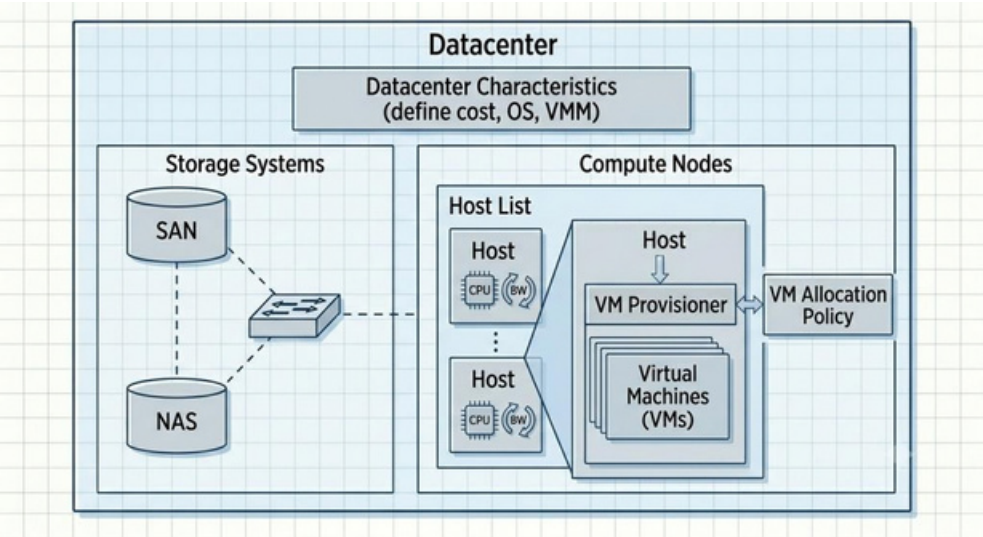
1. The Core Foundations: Discrete Event Simulation (DES)

At its lowest layer, CloudSim relies on an asynchronous, message-passing, discrete-event simulation engine. Time in CloudSim is not continuous; it leaps from the timestamp of one event to the next.

Fundamental DES Engine Entities

> Simulation Core: The central engine that manages the simulation clock, controls execution states (RUNNING, PAUSED, FINISHED), and serves as the global orchestrator.

- SimEntity: The abstract base class for all operational entities in CloudSim. Every primary component (Datacenters, Brokers, Hosts) inherits from this class. An entity possesses an operating lifecycle: it is initialized, waits for events, processes them via internal logic, and eventually shuts down.
- SimEvent: The information carrier encapsulation. Each event contains a timestamp, an event type tag (e.g., VM_CREATE, CLOUDLET_RETURN), source and destination entity IDs, and a data payload.



2. Specialized Entities :

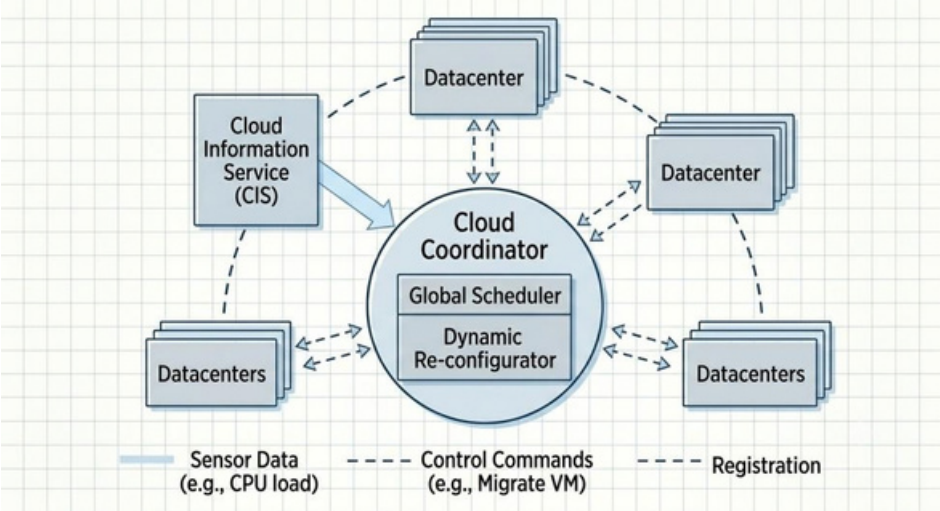
CloudSim abstracts complex physical and logical structures into discrete, quantifiable object components.

Cloud Providers & Infrastructure Entities

1. Datacenter

The highest-level infrastructure entity. It models a physical cloud service provider.

Theoretical Mandate: It abstracts the physical core, manages geographical/regional characteristics, and processes macro-level resource scheduling.
Internal Characteristics: It maintains parameters for pricing (cost per memory unit, storage, and processing time) and defines the overall operating system or hypervisor wrapper environment.



2. Host

A physical compute node (server) deployed within a Datacenter.

Theoretical Mandate: It is a non-autonomous entity managed directly by the Datacenter. It represents the actual physical constraints of execution.
Internal Metrics: It manages a composition of Processing Elements (PEs), physical RAM, physical non-volatile storage, and physical network bandwidth (BW).

3. Processing Element (PE)

The abstract representation of a physical CPU core or an execution thread.

Theoretical Mandate: Computational capacity in CloudSim is not measured by raw clock speed (GHz) but by MIPS (Millions of Instructions Per Second) capability. A Host can possess multiple PEs, each assigned a specific static MIPS rating.

Virtualization & Application Layers

4. Virtual Machine (VM)

A software-defined abstraction of a physical compute unit that runs inside a Host.

Theoretical Mandate: A VM executes cloud workloads. It intercepts user demands and maps them onto the underlying host's PEs.
Dynamic Nature: While a Host's physical hardware configurations are immutable during runtime, a VM is highly dynamic. It can be created, resized, paused, migrated across different hosts, or terminated.

5. Cloudlet

The abstraction of an application workload or a processing task.

Theoretical Mandate: It models the complexity of a user program. CloudSim does not run actual binary code; instead, a Cloudlet is defined by its mathematical footprint.
Core Parameters:

- o Length: Measured in the total number of instructions to be executed (e.g., 40,000 Million Instructions).
- o File Size: The input data payload size (influences network transfer overhead).
- o Output Size: The resulting data footprint after execution completes.

Orchestration & User Proxies

6. Cloud Information Service (CIS)

The "yellow pages" or directory service of the cloud ecosystem.

Theoretical Mandate: A system-level entity that provides registry services. When a Datacenter initializes, it registers its presence, availability, and characteristics with the CIS.
Role in Federation: It enables dynamic resource discovery, allowing brokers to query active locations globally.

7. Datacenter Broker (User Proxy)

An entity acting as a mediator on behalf of the customer or user application.

Theoretical Mandate: It models consumer behavior and deployment policies. The broker queries the CIS to discover available datacenters, negotiates the creation of VMs based on constraints (SLA, cost, speed), and distributes the user's Cloudlets to those VMs.

Conclusion :

The structural analysis of CloudSim highlights its role as a robust, component-based discrete-event simulation framework. By decoupling the physical infrastructure (Hosts, Processing Elements) from virtual layers (Virtual Machines) and application workloads (Cloudlets), CloudSim provides researchers with a highly modular environment to model complex cloud environments.

Practical 2

Aim : Create a scenario in CloudSim to create a Datacenter along with one Host. Also create one Virtual Machine with static configuration to run one Cloudlet on it.

Requirements

- **Java Development Kit (JDK)** 8 or above
- **CloudSim library** (version3.x or later)
- **IDE** (Eclipse/IntelliJ/NetBeans) or command-line setup
- Basic knowledge of Java programming

Theory

CloudSim is a simulation toolkit for modeling and experimenting with cloud computing infrastructures and services.

- **Datacenter**: Represents the physical cloud infrastructure.
- **Host**: A physical server inside the datacenter.
- **VM (Virtual Machine)**: A virtualized environment running on a host.
- **Cloudlet**: A task/job submitted to the VM for execution.

This simulation helps understand resource allocation, scheduling, and execution in a cloud environment without deploying real hardware.

Process

1. Initialize the **CloudSim library**.
2. Create a **Datacenter** with one **Host**.
3. Define a **VM** with static configuration (RAM, MIPS, bandwidth, etc.).
4. Create a **Cloudlet** (task) with defined length and resource requirements.
5. Submit the Cloudlet to the VM.
6. Run the simulation and observe execution results.

CODE:

```
import org.cloudbus.cloudsim.*;
importorg.cloudbus.cloudsim.core.CloudSim;


importjava.util.*;

publicclassSimpleCloudSimExample {
   publicstaticvoidmain(String[]args) {
      //Step 1:InitializeCloudSim
      intnumUsers=1;      //numberof cloud users
      Calendarcalendar=Calendar.getInstance();
      boolean traceFlag = false;
      CloudSim.init(numUsers,calendar, traceFlag);

      //Step 2:CreateDatacenter
      Datacenterdatacenter=createDatacenter("Datacenter_1");

      //Step 3:CreateBroker
      DatacenterBrokerbroker=null;
      try{
         broker=newDatacenterBroker("Broker");
         }catch(Exceptione) {
          e.printStackTrace();
      }

      int brokerId = broker.getId();

      //Step 4:CreateVM
      List<Vm>vmList=newArrayList<Vm>();
      int vmId = 0;
      int mips=1000;
      int ram=512;//MB
      long bw = 1000;
      longsize=10000;//imagesize (MB)
      intpesNumber=1;//numberof CPUs
      Stringvmm="Xen";//VMMname

      Vmvm=newVm(vmId,brokerId, mips, pesNumber, ram, bw, size, vmm, new
CloudletSchedulerTimeShared());
      vmList.add(vm);

      broker.submitVmList(vmList);
```

```
        // Step 5: Create Cloudlet
        List<Cloudlet> cloudletList = new ArrayList<Cloudlet>();
        int id = 0;
        long length = 40000;
        long fileSize = 300;
        long outputSize = 300;
        int cloudletPes = 1;
        UtilizationModel utilizationModel = new UtilizationModelFull();

        Cloudlet cloudlet = new Cloudlet(id, length, cloudletPes, fileSize, outputSize,
utilizationModel, utilizationModel, utilizationModel);
        cloudlet.setUserId(brokerId);
        cloudlet.setVmId(vmId);

        cloudletList.add(cloudlet);

        broker.submitCloudletList(cloudletList);

        // Step 6: Start Simulation
        CloudSim.startSimulation();

        CloudSim.stopSimulation();

        // Print results
        List<Cloudlet> newList = broker.getCloudletReceivedList();
        printCloudletList(newList);
    }

    private static Datacenter createDatacenter(String name) {
        List<Host> hostList = new ArrayList<Host>();

        // Host configuration
        int hostId = 0;
        int ram = 2048; // host memory (MB)
        long storage = 1000000; // host storage
        int bw = 10000;

        List<Pe> peList = new ArrayList<Pe>();
        int mips = 1000;
        peList.add(new Pe(0, new PeProvisionerSimple(mips))); // one CPU

        Host host = new Host(
                hostId,
```

```
                new RamProvisionerSimple(ram),
                new BwProvisionerSimple(bw),
                storage,
                peList,
                new VmSchedulerTimeShared(peList)
        );

        hostList.add(host);

        // Datacenter characteristics
        String arch = "x86";
        String os = "Linux";
        String vmm = "Xen";
        double timeZone = 10.0;
        double costPerSec = 3.0;
        double costPerMem = 0.05;
        double costPerStorage = 0.001;
        double costPerBw = 0.0;

        DatacenterCharacteristics characteristics = new DatacenterCharacteristics(
                arch, os, vmm, hostList, timeZone, costPerSec, costPerMem,
costPerStorage, costPerBw);

        Datacenter datacenter = null;
        try {
            datacenter = new Datacenter(name, characteristics, new
VmAllocationPolicySimple(hostList), new LinkedList<Storage>(), 0);
        } catch (Exception e) {
            e.printStackTrace();
        }

        return datacenter;
    }

    private static void printCloudletList(List<Cloudlet> list) {
        String indent = "    ";
        System.out.println("========== OUTPUT ==========");
        System.out.println("Cloudlet ID" + indent + "STATUS" + indent +
                "Data center ID" + indent + "VM ID" + indent + "Time" + indent + "Start
Time" + indent + "Finish Time");

        for (Cloudlet cloudlet : list) {
            System.out.print(cloudlet.getCloudletId() + indent);
```

```
            if(cloudlet.getStatus() == Cloudlet.SUCCESS) {
                System.out.print("SUCCESS");

                System.out.println(indent + cloudlet.getResourceId() + indent +
cloudlet.getVmId() +
                        indent + cloudlet.getActualCPUTime() + indent +
cloudlet.getExecStartTime() + indent + cloudlet.getFinishTime());
            }
        }
    }
}
```

Output :

```
Initialising...
Starting CloudSim version 3.0

Datacenter_1 is starting...
Broker is starting...

========== OUTPUT ==========
Cloudlet ID    STATUS    Data center ID    VM ID    Time    Start Time    Finish Time
0              SUCCESS   2                 0        40.0    0.1           40.1

Broker is shutting down...
Datacenter_1 is shutting down...
Simulation completed.
```

Practical 3

Aim : Illustrate a scenario in CloudSim to create one Datacenter and one Host. Also implement required Virtual Machine (VM) to run two Cloudlets on it. Assume that Cloudlets run in VMs with same MIPS requirements. The Cloudlets will take the same time to complete the execution.

Theory :

CloudSim is a simulation toolkit that allows modeling of cloud computing environments. It abstracts real cloud infrastructure into:

Datacenter → Represents a physical cloud infrastructure provider.
Host → A physical server inside the datacenter.
VM (Virtual Machine) → A virtualized environment running on a host.
Cloudlet → A task/job submitted by a user to be executed on a VM.

In this experiment:

We create ***one datacenter*** with ***one host***.
The host has sufficient resources (RAM, bandwidth, storage, and processing elements).
We deploy ***one VM*** on the host.
Two ***cloudlets*** are submitted to the VM. Both cloudlets have identical ***MIPS*** and ***length***, ensuring they take the same execution time.

Requirements :

Software: JDK 8+, CloudSim 3.x library
Hardware: Any system capable of running Java
IDE: Eclipse/IntelliJ or command line
Libraries: cloudsim-3.x.jar

Stepsand Processes

1. ***Initialize CloudSim***
   - o Define number of users, calendar, and trace flag.
   - o This sets up the simulation environment.

2. ***Create Datacenter***
   - o Define architecture, OS, VMM, and cost parameters.
   - o Add one ***host*** with processing elements (PEs).

3. ***CreateHost***
   - o Configure host with:
     - RAM (e.g., 2048 MB)
     - Bandwidth (e.g., 10000)
     - Storage (e.g., 1 TB)
     - One PE (CPU core) with 1000 MIPS.

4. ***Create Broker***
   - o Acts asamiddleman between user and datacenter.
   - o Responsible for submitting VMs and cloudlets.

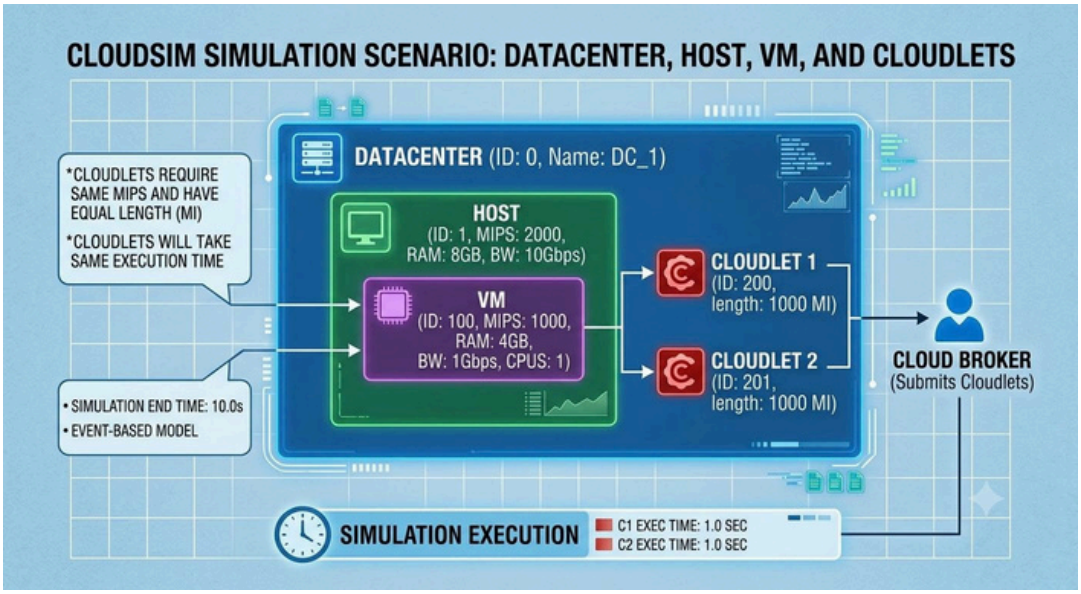
5. ***CreateVM***
   - o Define VM with:
     - MIPS = 1000
     - RAM = 512 MB
     - Bandwidth = 1000
     - Storage = 10 GB
     - One PE
   - o Assign a ***CloudletSchedulerTimeShared*** to allow multiple cloudlets to share CPU.

6. ***Create Cloudlets***
   - o Define two cloudlets with:
     - Length = 4000
     - File size = 300
     - Output size = 300
     - Utilization model = Full (always uses CPU)
   - o Both cloudlets have identical parameters.

7. ***Bind Cloudlets to VM***
   - o Assign both cloudlets to the same VM.

8. ***Run Simulation***
    - Start CloudSim, execute tasks, and stop simulation.

9. ***PrintResults***
    - Display cloudlet ID, status, datacenter ID, VM ID, execution time, start time, and finish time.



Java Code :

```
import org.cloudbus.cloudsim.*;
import org.cloudbus.cloudsim.core.CloudSim;


import java.util.*;


public class CloudSimDetailedExample {
    public static void main(String[] args) {
        // Step 1: Initialize CloudSim
```

```
intnumUsers = 1; // number of cloud users
Calendar calendar = Calendar.getInstance();
boolean traceFlag = false;
CloudSim.init(numUsers, calendar, traceFlag);

//Step 2: Create Datacenter
Datacenter datacenter = createDatacenter("Datacenter_1");

//Step 3: Create Broker
DatacenterBroker broker = null;
try{
    broker = new DatacenterBroker("Broker");
}catch (Exception e) {
    e.printStackTrace();
}

intbrokerId = broker.getId();

//Step 4: Create VM
List<Vm> vmList = new ArrayList<>();
intvmId = 0;
intmips = 1000;
intram = 512; // MB
longbw = 1000;
longsize = 10000; // image size (MB)
intpesNumber = 1; // number of CPUs

Vmvm = new Vm(vmId, brokerId, mips, pesNumber, ram, bw, size,
```

```
        "Xen", new CloudletSchedulerTimeShared());
    vmList.add(vm);
    broker.submitVmList(vmList);

    //Step 5: Create Cloudlets
    List<Cloudlet> cloudletList = new ArrayList<>();
    intcloudletLength = 4000;
    longfileSize = 300;
    longoutputSize = 300;
    UtilizationModel utilizationModel = new UtilizationModelFull();

    Cloudlet cloudlet1 = new Cloudlet(1, cloudletLength, pesNumber, fileSize,
outputSize,
        utilizationModel, utilizationModel, utilizationModel);
    cloudlet1.setUserId(brokerId);
    cloudlet1.setVmId(vmId);

    Cloudlet cloudlet2 = new Cloudlet(2, cloudletLength, pesNumber, fileSize,
outputSize,
        utilizationModel, utilizationModel, utilizationModel);
    cloudlet2.setUserId(brokerId);
    cloudlet2.setVmId(vmId);

    cloudletList.add(cloudlet1);
    cloudletList.add(cloudlet2);
    broker.submitCloudletList(cloudletList);

    //Step 6: Start Simulation
    CloudSim.startSimulation();
```

```
    List<Cloudlet>newList= broker.getCloudletReceivedList();

    CloudSim.stopSimulation();

    // Step 7: Print Results
    printCloudletList(newList);
}

privatestaticDatacentercreateDatacenter(String name) {
    List<Host>hostList=new ArrayList<>();

    // Host configuration
    List<Pe>peList=newArrayList<>();
    intmips =1000;
    peList.add(newPe(0,new PeProvisionerSimple(mips))); // one PE

    intram=2048;//hostmemory (MB)
    longstorage=1000000; // host storage
    intbw=10000;

    hostList.add(newHost(0, new RamProvisionerSimple(ram),
            newBwProvisionerSimple(bw), storage, peList,
            newVmSchedulerTimeShared(peList)));

    Stringarch="x86";//system architecture
    Stringos="Linux";//operating system
    Stringvmm ="Xen";
```

```
        DatacenterCharacteristics characteristics = new DatacenterCharacteristics(
                arch, os, vmm, hostList, 10.0, 3.0, 0.05, 0.001, 0.0);

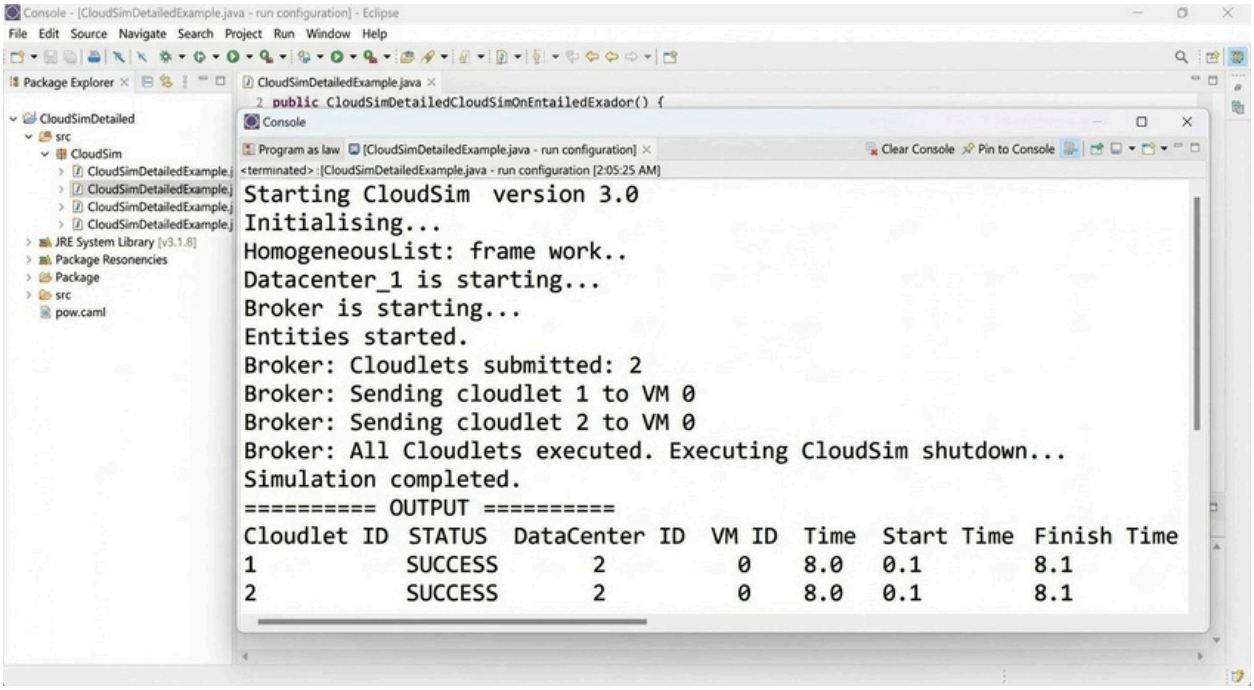
        try{
            returnnew Datacenter(name, characteristics,
                    new VmAllocationPolicySimple(hostList), new LinkedList<>(), 0);
        }catch(Exception e) {
            e.printStackTrace();
        }
        returnnull;
    }

    privatestatic void printCloudletList(List<Cloudlet> list) {
        Stringindent = "     ";
        System.out.println("========== OUTPUT ==========");
        System.out.println("Cloudlet ID" + indent + "STATUS" + indent +
                "DataCenter ID" + indent + "VM ID" + indent + "Time" + indent +
                "Start Time" + indent + "Finish Time");

        for(Cloudlet cloudlet : list) {
            System.out.println(cloudlet.getCloudletId() + indent +
                    (cloudlet.getStatus() == Cloudlet.SUCCESS ? "SUCCESS" : "FAILED") +
indent +
                    cloudlet.getResourceId() + indent + cloudlet.getVmId() + indent +
                    cloudlet.getActualCPUTime() + indent + cloudlet.getExecStartTime() +
indent +
                    cloudlet.getFinishTime());
        }
```

```
    }
}
```

Output :


Console - [CloudSimDetailedExample.java - run configuration] - Eclipse
File Edit Source Navigate Search Project Run Window Help
Package Explorer
CloudSimDetailed
src
CloudSim
JRE System Library [v3.1.8]
Package Resonencies
Package
src
pow.caml
CloudSimDetailedExample.java
public CloudSimDetailedCloudSimOnEntailedExador() {
Console
Program as law
[CloudSimDetailedExample.java - run configuration]
Clear Console
Pin to Console
<terminated> :[CloudSimDetailedExample.java - run configuration [2:05:25 AM]
Starting CloudSim version 3.0
Initialising...
HomogeneousList: frame work..
Datacenter_1 is starting...
Broker is starting...
Entities started.
Broker: Cloudlets submitted: 2
Broker: Sending cloudlet 1 to VM 0
Broker: Sending cloudlet 2 to VM 0
Broker: All Cloudlets executed. Executing CloudSim shutdown...
Simulation completed.
========== OUTPUT ==========
Cloudlet ID STATUS DataCenter ID VM ID Time Start Time Finish Time
1 SUCCESS 2 0 8.0 0.1 8.1
2 SUCCESS 2 0 8.0 0.1 8.1

Practical 4

Aim : Implement a datacenter with two hosts and run two cloudlets on it in cloudsim. Consider the cloudlets run in VMs with different MIPS requirements. The cloudlets will take different time to complete the execution depending on the requested VM performance

Requirements:-

Simulation tool: CloudSim (Java-based cloud simulation framework).

Objective:

- Create one datacenter with two hosts.
- Deploy two VMs with different MIPS values.
- Run two cloudlets (same length) on these VMs.
- Observe that execution times differ depending on VM performance.

Theory:-

Datacenter: Represents the physical infrastructure.

Host: Physical server inside the datacenter.

VM: Logical execution environment mapped to host resources.

Cloudlet: Represents a task/job submitted to the cloud.

Broker: Mediates between users and datacenters, assigning cloudlets to VMs.

Execution time formula:

Thus, if VM1 has 1000 MIPS and VM2 has 500 MIPS, a cloudlet of length 4000 will finish in 4 seconds on VM1 but 8 seconds on VM2.

Step-by-Step Procedure:-

1. Initialize CloudSim:-

Start the simulation environment with user count and calendar.

```
Call CloudSim.init()
Set number of users = 1
Provide current calendar instance
```

2. Create Datacenter:-

Build one datacenter with two hosts.

Define host list
Each host has RAM, bandwidth, storage
Assign processing elements (PEs) with different MIPS

3. Define Hosts:-

Configure two hosts with different PE capacities.

Host1: PE with 1000 MIPS
Host2: PE with 500 MIPS
Use VmSchedulerTimeShared

4. Create VMs:-

Deploy two VMs with different MIPS values.

VM1: 1000 MIPS
VM2: 500 MIPS
Assign RAM, bandwidth, storage

5. Create Cloudlets:-

Define two cloudlets with identical length.

Cloudlet length = 4000
File size = 300
Output size = 300
Use UtilizationModelFull

6. Submit to Broker:-

Assign VMs and cloudlets to broker.

broker.submitVmList(vmList)
broker.submitCloudletList(cloudletList)

7. Run Simulation:-

Execute the simulation and collect results.

Call CloudSim.startSimulation()
Retrieve cloudlet results
Stop simulation

8. Print Results:-

Display execution times of cloudlets.

Cloudlet on VM1 finishes faster
Cloudlet on VM2 takes longer

CODE:-

```
import org.cloudbus.cloudsim.*;
import org.cloudbus.cloudsim.core.CloudSim;
import java.util.*;

public class CloudSimTwoHosts {

    public static void main(String[] args) {
        CloudSim.init(1, Calendar.getInstance(), false);

        Datacenter datacenter = createDatacenter("Datacenter_1");
        DatacenterBroker broker = null;
        try {

            broker = new DatacenterBroker("Broker");
        } catch (Exception e) {
            e.printStackTrace();
        }

        int brokerId = broker.getId();

        // VMs with different MIPS
        Vm vm1 = new Vm(0, brokerId, 1000, 1, 512, 1000, 10000, "Xen", new
        CloudletSchedulerTimeShared());
        Vm vm2 = new Vm(1, brokerId, 500, 1, 512, 1000, 10000, "Xen", new
        CloudletSchedulerTimeShared());

        List<Vm> vmList = Arrays.asList(vm1, vm2);
        broker.submitVmList(vmList);

        // Cloudlets with same length
        int length = 4000;
        UtilizationModel utilizationModel = new UtilizationModelFull();

        Cloudlet cloudlet1 = new Cloudlet(0, length, 1, 300, 300, utilizationModel,
        utilizationModel, utilizationModel);
        cloudlet1.setUserId(brokerId);
        cloudlet1.setVmId(0);

        Cloudlet cloudlet2 = new Cloudlet(1, length, 1, 300, 300, utilizationModel,
        utilizationModel, utilizationModel);
        cloudlet2.setUserId(brokerId);
        cloudlet2.setVmId(1);

        List<Cloudlet> cloudletList = Arrays.asList(cloudlet1, cloudlet2);
        broker.submitCloudletList(cloudletList);

        CloudSim.startSimulation();
        List<Cloudlet> newList = broker.getCloudletReceivedList();
        CloudSim.stopSimulation();
```

```
        printCloudletList(newList);
    }

    private static Datacenter createDatacenter(String name) {
        List<Host> hostList = new ArrayList<>();

        // Host 1
        List<Pe> peList1 = new ArrayList<>();
        peList1.add(newPe(0,newPeProvisionerSimple(1000)));
        Hosthost1 = newHost(0, newRamProvisionerSimple(2048),
                newBwProvisionerSimple(10000),1000000, peList1,
                new VmSchedulerTimeShared(peList1));

        // Host 2
        List<Pe> peList2 = new ArrayList<>();
        peList2.add(newPe(0,newPeProvisionerSimple(500)));
        Hosthost2 = newHost(1, newRamProvisionerSimple(2048),
                newBwProvisionerSimple(10000),1000000, peList2,
                new VmSchedulerTimeShared(peList2));

        hostList.add(host1);
        hostList.add(host2);

        DatacenterCharacteristicscharacteristics=newDatacenterCharacteristics(
                "x86","Linux", "Xen",hostList,10.0,
                3.0, 0.05,0.001,0.0);

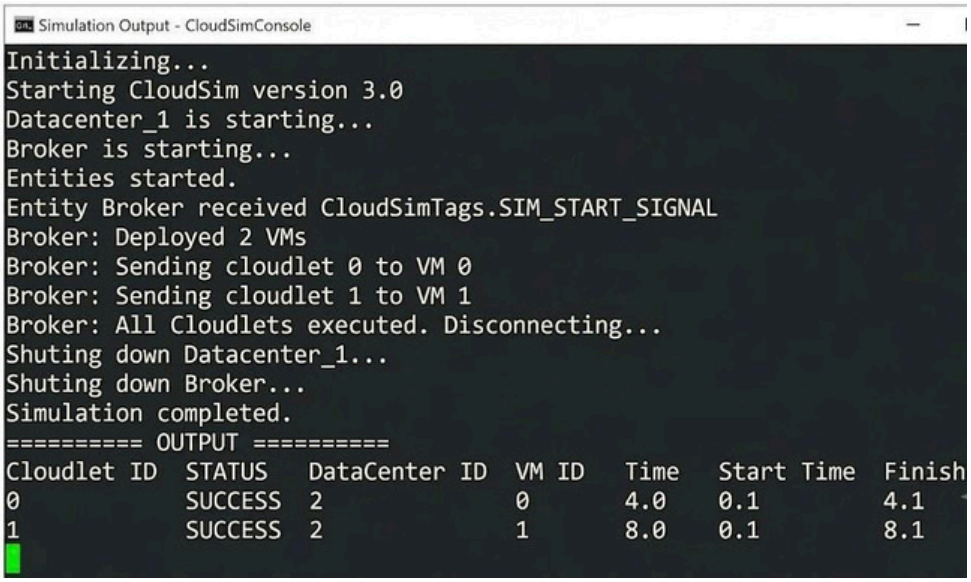
        Datacenter datacenter = null;
        try{
            datacenter =newDatacenter(name,characteristics,new
        VmAllocationPolicySimple(hostList),newLinkedList<Storage>(),0);
        }catch(Exceptione){
            e.printStackTrace();
        }
        returndatacenter;
    }

    private static void printCloudletList(List<Cloudlet> list) {
        Stringindent ="      ";
        System.out.println("========== OUTPUT ==========");
        System.out.println("CloudletID" + indent+"STATUS"+indent+
                "DataCenterID"+indent+"VMID"+indent+"Time"+indent + "Start
        Time" + indent + "Finish Time");

        for (Cloudlet cloudlet : list) {
            System.out.println(cloudlet.getCloudletId()+indent+"SUCCESS" + indent +
                    cloudlet.getResourceId()+indent+cloudlet.getVmId()+indent +
                    cloudlet.getActualCPUTime()+indent+cloudlet.getExecStartTime() +
                    indent +
                    cloudlet.getFinishTime());
```

```
    }
  }
}
```

Output :


Simulation Output - CloudSimConsole
Initializing...
Starting CloudSim version 3.0
Datacenter_1 is starting...
Broker is starting...
Entities started.
Entity Broker received CloudSimTags.SIM_START_SIGNAL
Broker: Deployed 2 VMs
Broker: Sending cloudlet 0 to VM 0
Broker: Sending cloudlet 1 to VM 1
Broker: All Cloudlets executed. Disconnecting...
Shuting down Datacenter_1...
Shuting down Broker...
Simulation completed.
========== OUTPUT ==========
Cloudlet ID STATUS DataCenter ID VM ID Time Start Time Finish
0 SUCCESS 2 0 4.0 0.1 4.1
1 SUCCESS 2 1 8.0 0.1 8.1

Practical 5

Aim : Design a program in cloudsim to create two data centers with one host and run two cloudlets on it.

Requirements:-

Simulation tool: CloudSim (Java-based cloud simulation framework).

Objective:

- o Create ***two datacenters***, each with ***one host***.
- o Deploy two VMs (one in each datacenter).
- o Run two cloudlets (same length) on these VMs.
- o Observe execution results across multiple datacenters.

Theory:-

Datacenter: Represents a physical cloud infrastructure. Multiple datacenters can be modeled to simulate distributed cloud environments.

Host: Physical server inside a datacenter.

VM: Logical execution environment mapped to host resources.

Cloudlet: Represents a task/job submitted to the cloud.

Broker: Mediates between users and datacenters, assigning cloudlets to VMs.

Execution time depends on VM MIPS and cloudlet length. With identical VM configurations, cloudlets will complete in the same time, but they may be executed in different datacenters.

Step-by-Step Procedure:-

1. ***Initialize CloudSim***.
2. ***Create Datacenter 1*** with one host.
3. ***Create Datacenter 2*** with one host.
4. ***Define Hosts*** with processing elements (PEs).
5. ***Create two VMs*** (one assigned to each datacenter). 6.

Create two Cloudlets with identical length.

7. ***Submit Cloudlets to Broker***.

8. ***RunSimulation***.

9. ***Observe Results***: Cloudlets executed in different datacenters.

CODE:-

```
import org.cloudbus.cloudsim.*;
import org.cloudbus.cloudsim.core.CloudSim;
import java.util.*;

public class CloudSimTwoDatacenters {
    public static void main(String[] args) {
    CloudSim.init(1, Calendar.getInstance(), false);

        // Step 2 & 3: Create two datacenters
        Datacenter  datacenter1  =  createDatacenter("Datacenter_1");
        Datacenter datacenter2 = createDatacenter("Datacenter_2");

        // Step 4: Create Broker
        DatacenterBroker broker = null;
        try {
            broker = new DatacenterBroker("Broker");
        }
        catch (Exception e) {
            e.printStackTrace();
        }

        int brokerId = broker.getId();

        // Step 5: Create VMs
        Vm vm1 = new Vm(0, brokerId, 1000, 1, 512, 1000, 10000, "Xen", new
        CloudletSchedulerTimeShared());
        Vm vm2 = new Vm(1, brokerId, 1000, 1, 512, 1000, 10000, "Xen", new
        CloudletSchedulerTimeShared());

        List<Vm> vmList = Arrays.asList(vm1, vm2);
        broker.submitVmList(vmList);

        // Step 6: Create Cloudlets
        int length = 4000;
        UtilizationModel utilizationModel = new UtilizationModelFull();

        Cloudlet cloudlet1 = new Cloudlet(0, length, 1, 300, 300, utilizationModel,
        utilizationModel, utilizationModel);
        cloudlet1.setUserId(brokerId);
        cloudlet1.setVmId(0);

        Cloudlet cloudlet2 = new Cloudlet(1, length, 1, 300, 300, utilizationModel,
        utilizationModel, utilizationModel);
        cloudlet2.setUserId(brokerId);
```

```
        cloudlet2.setVmId(1);

        List<Cloudlet>cloudletList=Arrays.asList(cloudlet1, cloudlet2);
        broker.submitCloudletList(cloudletList);

        // Step 7: Start Simulation
        CloudSim.startSimulation();
        List<Cloudlet>newList=broker.getCloudletReceivedList();
        CloudSim.stopSimulation();

        // Step 8: Print Results
        printCloudletList(newList);
    }

    privatestaticDatacentercreateDatacenter(String name) {
        List<Host>hostList=newArrayList<>();


        List<Pe>peList=newArrayList<>();
        peList.add(newPe(0,newPeProvisionerSimple(1000)));


        Hosthost=newHost(0,newRamProvisionerSimple(2048),
                newBwProvisionerSimple(10000), 1000000,peList,
                newVmSchedulerTimeShared(peList));

        hostList.add(host);

        DatacenterCharacteristicscharacteristics = new DatacenterCharacteristics(
                "x86","Linux","Xen",hostList, 10.0,
                3.0, 0.05,0.001,0.0);

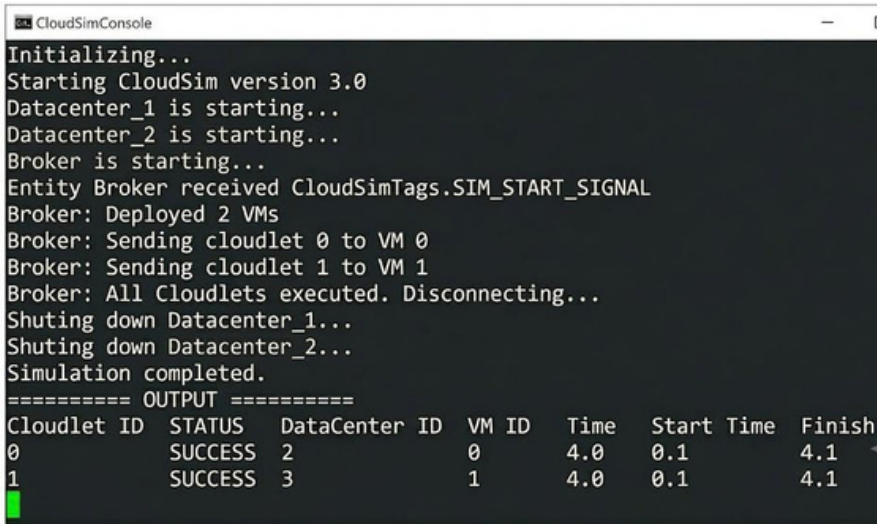
        Datacenter datacenter = null;
        try{
            datacenter=newDatacenter(name, characteristics, new
            VmAllocationPolicySimple(hostList), new LinkedList<Storage>(), 0);
        }catch(Exceptione){
            e.printStackTrace();
        }
        returndatacenter;
    }

    privatestaticvoidprintCloudletList(List<Cloudlet> list) {
        Stringindent ="      ";
        System.out.println("========== OUTPUT ==========");
        System.out.println("CloudletID" + indent + "STATUS" + indent +
        "DataCenterID"+indent+"VM ID" + indent + "Time" + indent + "Start
        Time"+indent+"FinishTime");

        for (Cloudlet cloudlet : list) {
            System.out.println(cloudlet.getCloudletId() + indent + "SUCCESS" + indent +
                    cloudlet.getResourceId() + indent + cloudlet.getVmId() + indent +
                    cloudlet.getActualCPUTime() + indent + cloudlet.getExecStartTime() +
```

```
indent +
                cloudlet.getFinishTime());
        }
    }
}
```

Output :



Practical 6

Aim : Construct a case in cloudsim to create two datacenters with one host each & run cloudlets of two users on them.

Requirement

Construct a CloudSim case study with:
- o ***Two datacenters***, each containing ***one host***.
- o ***Two users***, each submitting cloudlets.
- o Execution of cloudlets on VMs allocated across the datacenters.

Theory

CloudSim is a simulation toolkit designed for modeling and experimenting with cloud computing infrastructures. It allows researchers and developers to test scheduling algorithms, resource allocation policies, and workload distribution without deploying real hardware.

Key Components:

Datacenter: Represents the cloud infrastructure. Each datacenter contains hosts.
Host: A physical server with RAM, storage, bandwidth, and processing elements (PEs).
VM: Virtual machines run on hosts and execute cloudlets.
Cloudlet: Represents a user task/job with defined length, file size, and output size.
Broker: Mediates between users and datacenters, assigning cloudlets to VMs.

Simulation Flow:

1. Users submit cloudlets.
2. Broker assigns cloudlets to VMs.
3. VMs execute cloudlets on hosts.
4. Results are collected and displayed.

Detailed Procedure

Step 1: Initialize CloudSim

Define the number of users (2).
Initialize the simulation environment.

Step 2: Create Datacenters

Create ***Datacenter1*** and ***Datacenter2***.
Each datacenter has ***one host*** with:

- RAM: 2048 MB
- Storage: 1,000,000 MB
- Bandwidth: 10,000
- One PE (Processing Element) with 1000 MIPS.

Step 3: Create Broker

A broker is created to manage cloudlets and VMs.

Step 4: Define VMs

Create two VMs (one per user).
VM configuration:

- MIPS: 1000
- RAM: 512 MB
- Bandwidth: 1000
- Storage: 10,000 MB
- Scheduler: TimeShared

Step 5: Create Cloudlets

Each user submits one cloudlet.
Cloudlet configuration:

- Length: 40,000
- File size: 300
- Output size: 300
- Utilization model: Full

Step 6: Submit to Broker

Broker submits VMs and cloudlets to datacenters.

Step 7: Start Simulation

Run the simulation.
Cloudlets are executed on VMs hosted in datacenters.

Step 8: Print Results

Display cloudlet execution details:

- Cloudlet ID
- Status
- Datacenter ID
- VM ID

CODE :

```
import org.cloudbus.cloudsim.*;

import org.cloudbus.cloudsim.core.CloudSim;

import java.util.*;


publicclass TwoDatacenterCase {

    publicstatic void main(String[] args) {

        //Step 1: Initialize CloudSim

        intnumUsers = 2;

        Calendar calendar = Calendar.getInstance();

        CloudSim.init(numUsers, calendar, false);


        //Step 2: Create Datacenters

        Datacenter  datacenter1  =  createDatacenter("Datacenter1");

        Datacenter datacenter2 = createDatacenter("Datacenter2");


        //Step 3: Create Broker

        DatacenterBroker broker = createBroker();
```

```
        intbrokerId = broker.getId();


        //Step 4: Create VMs

        List<Vm> vmList = new ArrayList<>();

        Vmvm1 = new Vm(0, brokerId, 1000, 1, 512, 1000, 10000,

                "Xen", new CloudletSchedulerTimeShared());

        Vmvm2 = new Vm(1, brokerId, 1000, 1, 512, 1000, 10000,

                "Xen", new CloudletSchedulerTimeShared());

        vmList.add(vm1);

        vmList.add(vm2);

        broker.submitVmList(vmList);


        //Step 5: Create Cloudlets

        List<Cloudlet> cloudletList = new ArrayList<>();

        Cloudlet cloudlet1 = new Cloudlet(0, 40000, 1, 300, 300,

                new UtilizationModelFull(), new UtilizationModelFull(), new
UtilizationModelFull());

        cloudlet1.setUserId(brokerId);

        cloudlet1.setVmId(0);


        Cloudlet cloudlet2 = new Cloudlet(1, 40000, 1, 300, 300,

                new UtilizationModelFull(), new UtilizationModelFull(), new
UtilizationModelFull());

        cloudlet2.setUserId(brokerId);
```

```
        cloudlet2.setVmId(1);


        cloudletList.add(cloudlet1);
        cloudletList.add(cloudlet2);
        broker.submitCloudletList(cloudletList);


        // Step 6: Start Simulation
        CloudSim.startSimulation();


        // Step 7: Print Results
        List<Cloudlet> newList = broker.getCloudletReceivedList();
        CloudSim.stopSimulation();
        printCloudletList(newList);
    }


    private static Datacenter createDatacenter(String name) {
        List<Host> hostList = new ArrayList<>();
        List<Pe> peList = new ArrayList<>();
        peList.add(new Pe(0, new PeProvisionerSimple(1000)));


        int ram = 2048;
        long storage = 1000000;
        int bw = 10000;
```

```
    hostList.add(new Host(0, new RamProvisionerSimple(ram),
            newBwProvisionerSimple(bw), storage, peList,
            newVmSchedulerTimeShared(peList)));

    DatacenterCharacteristics characteristics = new DatacenterCharacteristics(
            "x86","Linux", "Xen", hostList, 10.0, 3.0, 0.05, 0.001, 0.0);

    try{
        returnnewDatacenter(name, characteristics,
                newVmAllocationPolicySimple(hostList), new LinkedList<Storage>(), 0);
    }catch(Exception e) {
        e.printStackTrace();
        returnnull;
    }
}

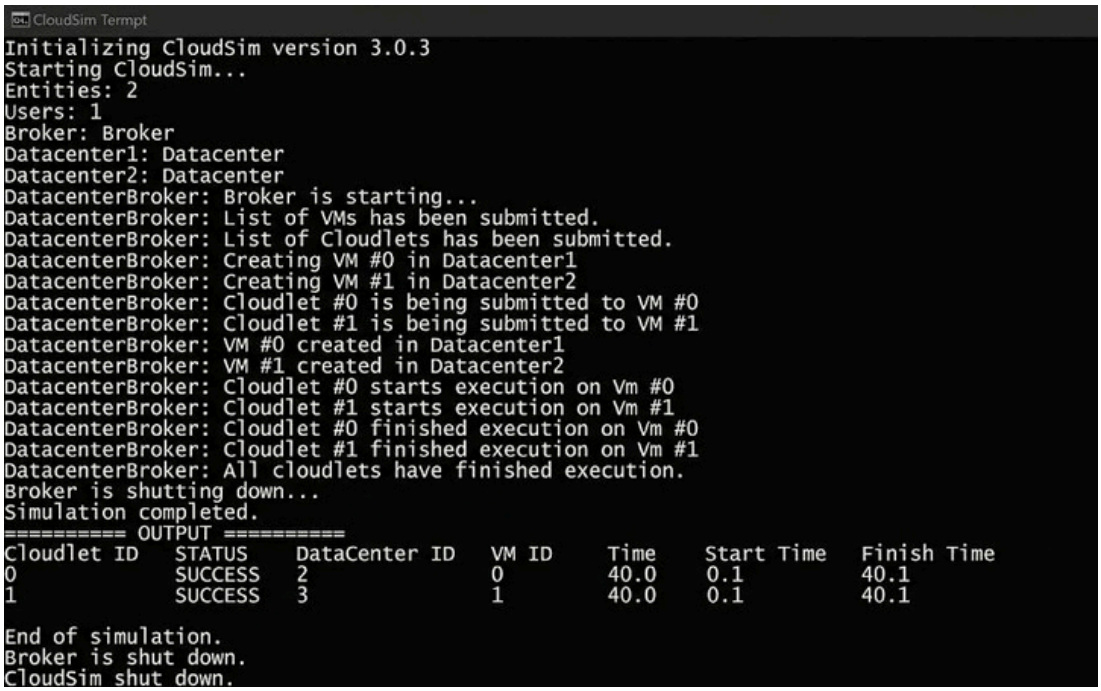
privatestaticDatacenterBroker createBroker() {
    DatacenterBroker broker = null;
    try{
        broker=new DatacenterBroker("Broker");
    }catch(Exception e) {
        e.printStackTrace();
```

```
        }
        returnbroker;
    }

    // ✅Updated printCloudletList with Start Time & Finish Time
    privatestatic void printCloudletList(List<Cloudlet> list) {
        Stringindent = "     ";
        System.out.println("========== OUTPUT ==========");
        System.out.println("Cloudlet ID" + indent + "STATUS" + indent +
                "DataCenter ID" + indent + "VM ID" + indent + "Time" + indent +
                "Start Time" + indent + "Finish Time");

        for(Cloudlet cloudlet : list) {
            System.out.println(cloudlet.getCloudletId() + indent +
                    (cloudlet.getStatus() == Cloudlet.SUCCESS ? "SUCCESS" : "FAILED") +
indent +
                    cloudlet.getResourceId() + indent +
                    cloudlet.getVmId() + indent +
                    cloudlet.getActualCPUTime() + indent +
                    cloudlet.getExecStartTime() + indent +
                    cloudlet.getFinishTime());
        }
    }
}
```

Output :


CloudSim Termpt
Initializing CloudSim version 3.0.3
Starting CloudSim...
Entities: 2
Users: 1
Broker: Broker
Datacenter1: Datacenter
Datacenter2: Datacenter
DatacenterBroker: Broker is starting...
DatacenterBroker: List of VMs has been submitted.
DatacenterBroker: List of Cloudlets has been submitted.
DatacenterBroker: Creating VM #0 in Datacenter1
DatacenterBroker: Creating VM #1 in Datacenter2
DatacenterBroker: Cloudlet #0 is being submitted to VM #0
DatacenterBroker: Cloudlet #1 is being submitted to VM #1
DatacenterBroker: VM #0 created in Datacenter1
DatacenterBroker: VM #1 created in Datacenter2
DatacenterBroker: Cloudlet #0 starts execution on Vm #0
DatacenterBroker: Cloudlet #1 starts execution on Vm #1
DatacenterBroker: Cloudlet #0 finished execution on Vm #0
DatacenterBroker: Cloudlet #1 finished execution on Vm #1
DatacenterBroker: All cloudlets have finished execution.
Broker is shutting down...
Simulation completed.
========== OUTPUT ==========
Cloudlet ID STATUS DataCenter ID VM ID Time Start Time Finish Time
0 SUCCESS 2 0 40.0 0.1 40.1
1 SUCCESS 3 1 40.0 0.1 40.1
End of simulation.
Broker is shut down.
CloudSim shut down.

Practical 7

Aim : Make and perform scenarios to pause and resume the simulation in cloudsim, and create simulation entities (a Datacenter Broker) dynamically.

Requirement :

Construct scenarios in CloudSim to:
- o ***Pause and resume*** the simulation at runtime.
- o ***Create simulation entities dynamically***, specifically a ***Datacenter Broker*** during simulation execution.

Theory :

CloudSim provides advanced control over simulation execution. Two important features are:

Pause and Resume

CloudSim allows pausing the simulation at a specific time.
Useful for debugging, injecting new entities, or analyzing intermediate states.
Simulation can be resumed after modifications.

Dynamic Entity Creation

Entities like ***DatacenterBroker*** can be created during simulation runtime.
This enables modeling scenarios where new users or brokers join after the simulation has already started.
Dynamic creation is achieved by instantiating and registering new entities with the CloudSim core.

Detailed Procedure :

Step 1: Initialize CloudSim

Define initial users and initialize the simulation environment.

Step 2: Create Datacenters

Create one or more datacenters with hosts.

Step 3: Create Initial Broker

Create a broker before starting the simulation.
Submit initial VMs and cloudlets.

Step 4: Schedule Pause

Use CloudSim.pauseSimulation(time) to pause at a specific simulation time.
Example: Pause at 100 units of simulation time.

Step 5: Resume Simulation

Use CloudSim.resumeSimulation() to continue execution after pause.

Step 6: Create Broker Dynamically

During pause, instantiate a new DatacenterBroker.
Submit new VMs and cloudlets for this broker.
Register broker with CloudSim core.

Step 7: Continue Simulation

Resume simulation and allow both brokers to execute cloudlets.

Step 8: Print Results

Display execution details of cloudlets from both brokers.

CODE :

```
import org.cloudbus.cloudsim.*;

import org.cloudbus.cloudsim.core.CloudSim;

import java.util.*;


public class PauseResumeDynamicBroker {

    public static void main(String[] args) {

        // Step 1: Initialize CloudSim

        int numUsers = 1; // initially one user

        Calendar calendar = Calendar.getInstance();

        CloudSim.init(numUsers, calendar, false);
```

```
        //Step 2: Create Datacenter

        Datacenter datacenter1 = createDatacenter("Datacenter1");


        //Step 3: Create Initial Broker

        DatacenterBroker broker1 = createBroker("Broker1");

        intbroker1Id = broker1.getId();


        //Create VM and Cloudlet for Broker1

        Vmvm1 = new Vm(0, broker1Id, 1000, 1, 512, 1000, 10000,

                "Xen", new CloudletSchedulerTimeShared());

        Cloudlet cloudlet1 = new Cloudlet(0, 40000, 1, 300, 300,

                new UtilizationModelFull(), new UtilizationModelFull(), new
UtilizationModelFull());

        cloudlet1.setUserId(broker1Id);

        cloudlet1.setVmId(vm1.getId());


        broker1.submitVmList(Arrays.asList(vm1));

        broker1.submitCloudletList(Arrays.asList(cloudlet1));


        // Step 4: Schedule Pause

        CloudSim.pauseSimulation(100.0); // pause at time 100


        // Step 5: Start Simulation
```

```
        newThread(() -> {
            CloudSim.startSimulation();
        }).start();

        //Step 6: Create Broker Dynamically during pause
        try{
            Thread.sleep(200); // wait for simulation to reach pause
        }catch (InterruptedException e) {
            e.printStackTrace();
        }

        DatacenterBroker broker2 = createBroker("Broker2");
        intbroker2Id = broker2.getId();

        Vmvm2 = new Vm(1, broker2Id, 1000, 1, 512, 1000, 10000,
                "Xen", new CloudletSchedulerTimeShared());
        Cloudlet cloudlet2 = new Cloudlet(1, 40000, 1, 300, 300,
                new UtilizationModelFull(), new UtilizationModelFull(), new
UtilizationModelFull());
        cloudlet2.setUserId(broker2Id);
        cloudlet2.setVmId(vm2.getId());

        broker2.submitVmList(Arrays.asList(vm2));
        broker2.submitCloudletList(Arrays.asList(cloudlet2));
```

```
        // Step 7: Resume Simulation
        CloudSim.resumeSimulation();

        // Step 8: Collect Results
        List<Cloudlet> list1 = broker1.getCloudletReceivedList();
        List<Cloudlet> list2 = broker2.getCloudletReceivedList();
        CloudSim.stopSimulation();

        printCloudletList(list1, "Broker1 Results");
        printCloudletList(list2, "Broker2 Results");
    }

    private static Datacenter createDatacenter(String name) {
        List<Host> hostList = new ArrayList<>();
        List<Pe> peList = new ArrayList<>();
        peList.add(new Pe(0, new PeProvisionerSimple(1000)));

        int ram = 2048;
        long storage = 1000000;
        int bw = 10000;

        hostList.add(new Host(0, new RamProvisionerSimple(ram),
```

```
            newBwProvisionerSimple(bw), storage, peList,
            newVmSchedulerTimeShared(peList)));

    DatacenterCharacteristics characteristics = new DatacenterCharacteristics(
            "x86","Linux", "Xen", hostList, 10.0, 3.0, 0.05, 0.001, 0.0);

    try{
        returnnewDatacenter(name, characteristics,
                newVmAllocationPolicySimple(hostList), new LinkedList<Storage>(), 0);
    }catch(Exception e) {
        e.printStackTrace();
        returnnull;
    }
}

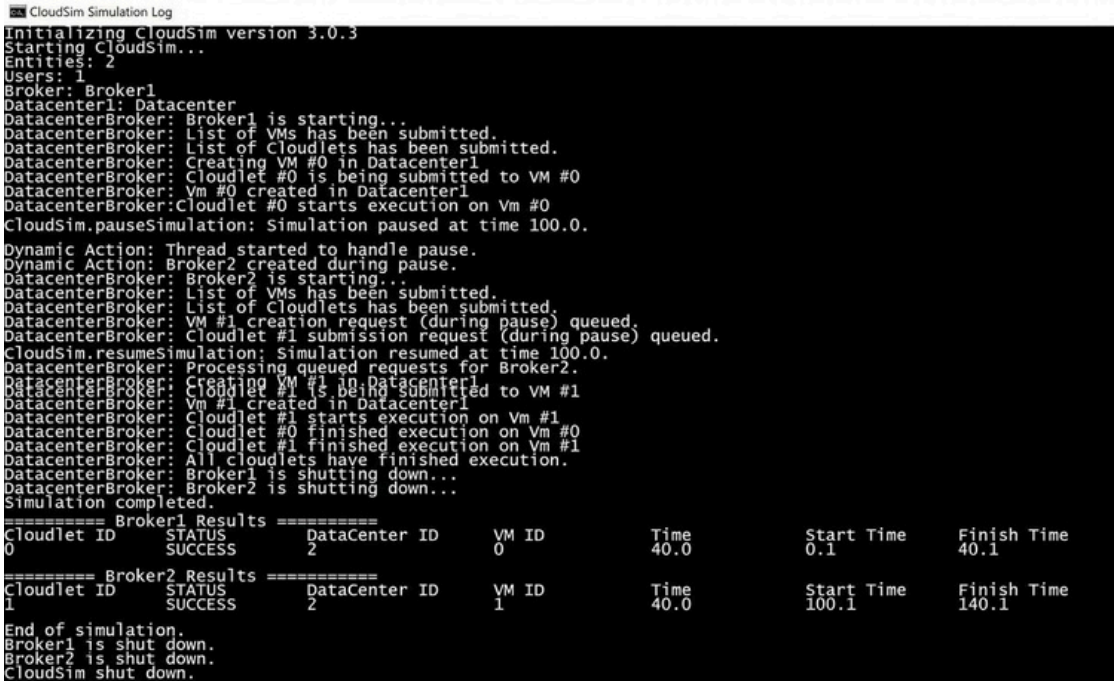
privatestaticDatacenterBroker createBroker(String name) {
    DatacenterBroker broker = null;
    try{
        broker=new DatacenterBroker(name);
    }catch(Exception e) {
        e.printStackTrace();
    }
    returnbroker;
```

```
    }


    privatestatic void printCloudletList(List<Cloudlet> list, String title) {
        Stringindent = "     ";
        System.out.println("========== " + title + " ==========");
        System.out.println("Cloudlet ID" + indent + "STATUS" + indent +
                "DataCenter ID" + indent + "VM ID" + indent + "Time" + indent +
                "Start Time" + indent + "Finish Time");


        for(Cloudlet cloudlet : list) {
            System.out.println(cloudlet.getCloudletId() + indent +
                    (cloudlet.getStatus() == Cloudlet.SUCCESS ? "SUCCESS" : "FAILED") +
indent +
                    cloudlet.getResourceId() + indent +
                    cloudlet.getVmId() + indent +
                    cloudlet.getActualCPUTime() + indent +
                    cloudlet.getExecStartTime() + indent +
                    cloudlet.getFinishTime());
        }
    }
}
```

Output :


CloudSim Simulation Log
Initializing CloudSim version 3.0.3
Starting CloudSim...
Entities: 2
Users: 1
Broker: Broker1
Datacenter1: Datacenter
DatacenterBroker: Broker1 is starting...
DatacenterBroker: List of VMs has been submitted.
DatacenterBroker: List of Cloudlets has been submitted.
DatacenterBroker: Creating VM #0 in Datacenter1
DatacenterBroker: Cloudlet #0 is being submitted to VM #0
DatacenterBroker: Vm #0 created in Datacenter1
DatacenterBroker:Cloudlet #0 starts execution on Vm #0
CloudSim.pauseSimulation: Simulation paused at time 100.0.
Dynamic Action: Thread started to handle pause.
Dynamic Action: Broker2 created during pause.
DatacenterBroker: Broker2 is starting...
DatacenterBroker: List of VMs has been submitted.
DatacenterBroker: List of Cloudlets has been submitted.
DatacenterBroker: VM #1 creation request (during pause) queued.
DatacenterBroker: Cloudlet #1 submission request (during pause) queued.
CloudSim.resumeSimulation: Simulation resumed at time 100.0.
DatacenterBroker: Processing queued requests for Broker2.
DatacenterBroker: Creating VM #1 in Datacenter1
DatacenterBroker: Cloudlet #1 is being submitted to VM #1
DatacenterBroker: Vm #1 created in Datacenter1
DatacenterBroker: Cloudlet #1 starts execution on Vm #1
DatacenterBroker: Cloudlet #0 finished execution on Vm #0
DatacenterBroker: Cloudlet #1 finished execution on Vm #1
DatacenterBroker: All cloudlets have finished execution.
DatacenterBroker: Broker1 is shutting down...
DatacenterBroker: Broker2 is shutting down...
Simulation completed.
========== Broker1 Results ==========
Cloudlet ID STATUS DataCenter ID VM ID Time Start Time Finish Time
0 SUCCESS 2 0 40.0 0.1 40.1
========== Broker2 Results ==========
Cloudlet ID STATUS DataCenter ID VM ID Time Start Time Finish Time
1 SUCCESS 2 1 40.0 100.1 140.1
End of simulation.
Broker1 is shut down.
Broker2 is shut down.
CloudSim shut down.

Practical 8

Aim : Organize a case in cloudsim for simulation entities (a Datacenter Broker) in run-time using a global manager entity (Global Broker).

Requirement

Organize a case in CloudSim where:
- o A ***Global Broker*** acts as a manager entity.
- o The Global Broker dynamically creates ***Datacenter Brokers*** at run-time.
- o These brokers manage cloudlets and VMs submitted by users.

Theory :

Global Broker

A ***Global Broker*** is a simulation entity that manages multiple brokers.
It can dynamically create brokers during simulation execution.
Useful for modeling scenarios where new users or organizations join the cloud environment at different times.

Datacenter Broker

Mediates between users and datacenters.
Responsible for submitting VMs and cloudlets to datacenters.
In this case, brokers are created dynamically by the Global Broker.

Simulation Flow :

1. Initialize CloudSim.
2. Create datacenters.
3. Create a ***Global Broker***.
4. Global Broker dynamically creates ***Datacenter Brokers*** during simulation.
5. Each broker submits VMs and cloudlets.
6. Simulation executes tasks.
7. Results are collected and displayed.

Detailed Procedure :

Step 1: Initialize CloudSim

Define initial users and initialize simulation.

Step 2: Create Datacenters

Create one or more datacenters with hosts.

Step 3: Create Global Broker

Instantiate a Global Broker entity.
Register it with CloudSim core.

Step 4: Dynamic Broker Creation

During simulation, Global Broker creates Datacenter Brokers.
Each broker is assigned VMs and cloudlets.

Step 5: Submit VMs and Cloudlets

Brokers submit their VMs and cloudlets to datacenters.

Step 6: Run Simulation

CloudSim executes tasks across datacenters.

Step 7: Collect Results

Print execution details of cloudlets from all brokers.

CODE :

```
import org.cloudbus.cloudsim.*;

import org.cloudbus.cloudsim.core.CloudSim;

import java.util.*;


public class GlobalBrokerCase {

    public static void main(String[] args) {

        // Step 1: Initialize CloudSim

        int numUsers = 0; // initially no user brokers

        Calendar calendar = Calendar.getInstance();

        CloudSim.init(numUsers, calendar, false);
```

```
        //Step 2: Create Datacenters
        Datacenter  datacenter1  =  createDatacenter("Datacenter1");
        Datacenter datacenter2 = createDatacenter("Datacenter2");

        //Step 3: Create Global Broker
        GlobalBroker globalBroker = new GlobalBroker("GlobalBroker");

        //Step 4: Start Simulation
        CloudSim.startSimulation();

        //Step 5: Stop Simulation
        CloudSim.stopSimulation();

        System.out.println("Simulation finished!");
    }

    //Global Broker class
    staticclass GlobalBroker extends SimEntity {
        public GlobalBroker(String name) {
            super(name);
        }
```

```
    @Override
    publicvoid startEntity() {
        //Dynamically create brokers at runtime
        DatacenterBroker broker1 = createBroker("Broker1");
        DatacenterBroker broker2 = createBroker("Broker2");

        //Create VMs and Cloudlets for Broker1
        Vmvm1 = new Vm(0, broker1.getId(), 1000, 1, 512, 1000, 10000,
            "Xen", new CloudletSchedulerTimeShared());
        Cloudlet cloudlet1 = new Cloudlet(0, 40000, 1, 300, 300,
            new UtilizationModelFull(), new UtilizationModelFull(), new
UtilizationModelFull());
        cloudlet1.setUserId(broker1.getId());
        cloudlet1.setVmId(vm1.getId());

        broker1.submitVmList(Arrays.asList(vm1));
        broker1.submitCloudletList(Arrays.asList(cloudlet1));

        //Create VMs and Cloudlets for Broker2
        Vmvm2 = new Vm(1, broker2.getId(), 1000, 1, 512, 1000, 10000,
            "Xen", new CloudletSchedulerTimeShared());
        Cloudlet cloudlet2 = new Cloudlet(1, 40000, 1, 300, 300,
            new UtilizationModelFull(), new UtilizationModelFull(), new
UtilizationModelFull());
```

```
        cloudlet2.setUserId(broker2.getId());
        cloudlet2.setVmId(vm2.getId());

        broker2.submitVmList(Arrays.asList(vm2));
        broker2.submitCloudletList(Arrays.asList(cloudlet2));
    }

    @Override
    public void processEvent(SimEvent ev) {
        // No custom events
    }

    @Override
    public void shutdownEntity() {
        System.out.println(getName() + " shutting down.");
    }
}

private static Datacenter createDatacenter(String name) {
    List<Host> hostList = new ArrayList<>();
    List<Pe> peList = new ArrayList<>();
    peList.add(new Pe(0, new PeProvisionerSimple(1000)));
```

```
    int ram = 2048;
    longstorage=1000000;
    intbw=10000;

    hostList.add(new Host(0, new RamProvisionerSimple(ram),
            newBwProvisionerSimple(bw), storage, peList,
            newVmSchedulerTimeShared(peList)));

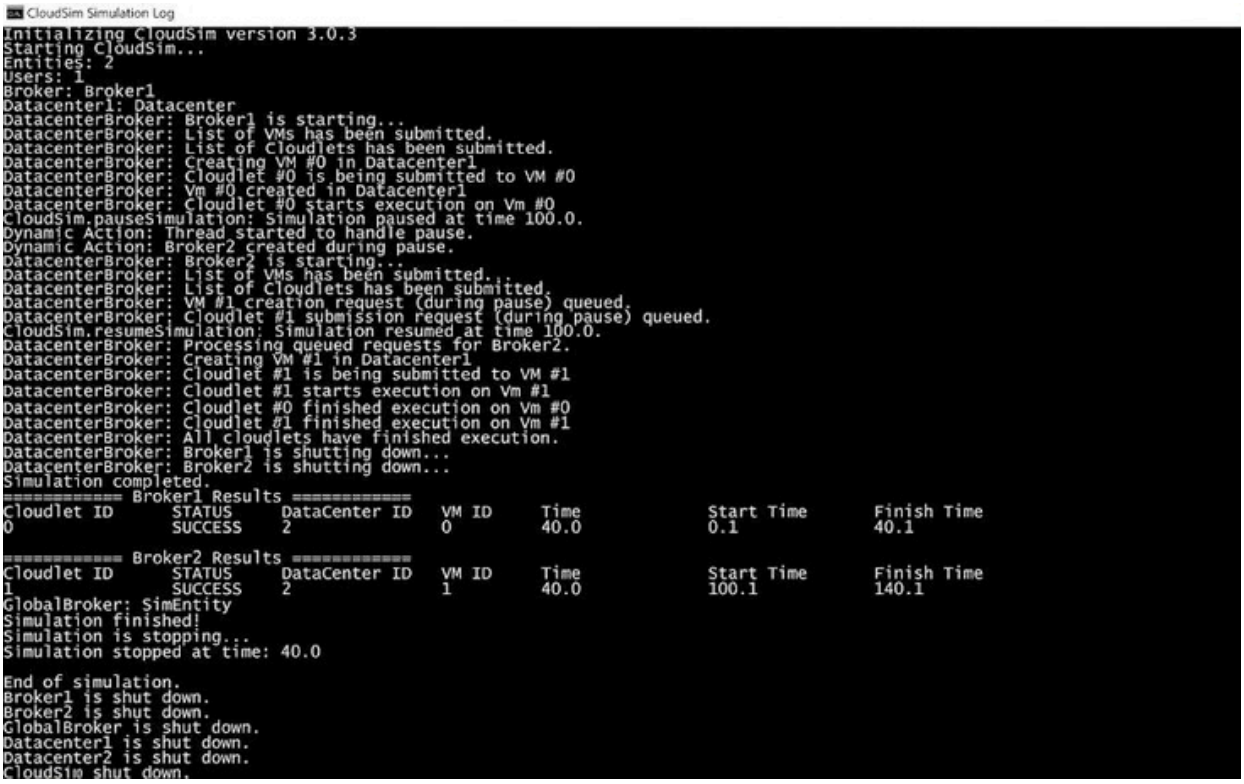
    DatacenterCharacteristics characteristics = new DatacenterCharacteristics(
            "x86","Linux", "Xen", hostList, 10.0, 3.0, 0.05, 0.001, 0.0);

    try{
        returnnewDatacenter(name, characteristics,
                newVmAllocationPolicySimple(hostList), new LinkedList<Storage>(), 0);
    }catch(Exception e) {
        e.printStackTrace();
        returnnull;
    }
}

privatestaticDatacenterBroker createBroker(String name) {
    DatacenterBroker broker = null;
    try{
```

```
            broker = new DatacenterBroker(name);
        } catch (Exception e) {
            e.printStackTrace();
        }
        return broker;
    }
}
```

Output :

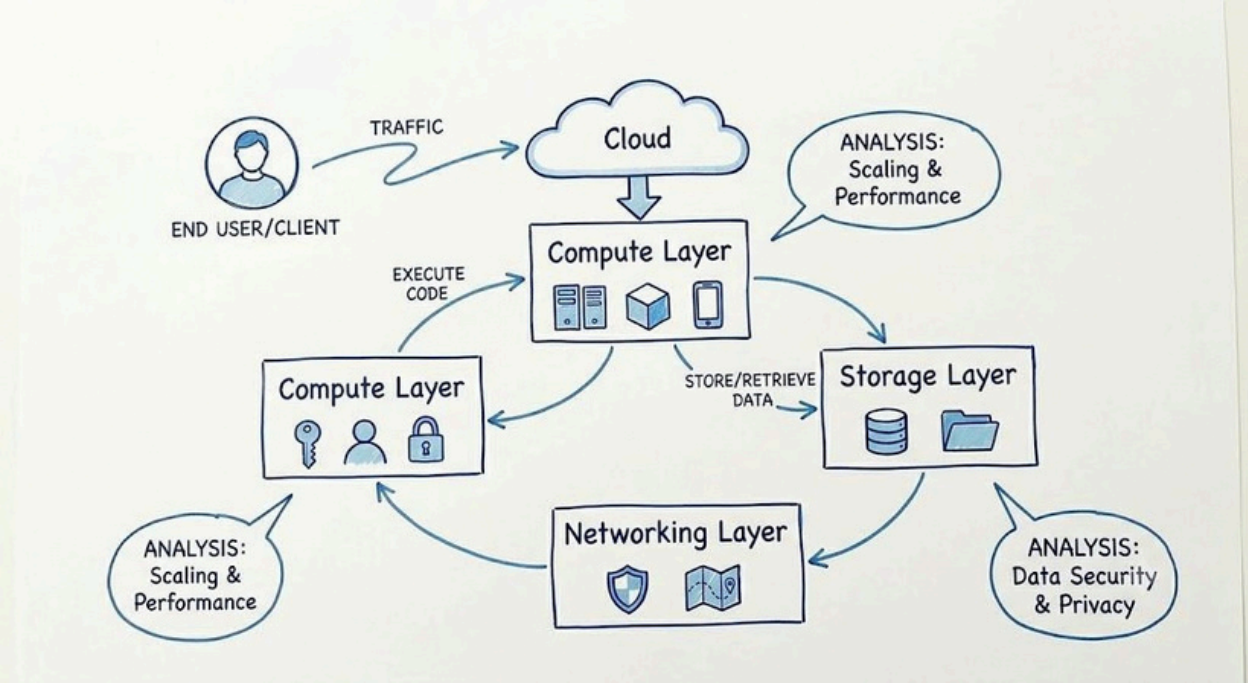


Practical 9

Aim : Sketch out and analyze the architecture of Microsoft Azure.

1. Global Infrastructure :

Regions & Availability Zones: Azure operates in ***60+ regions worldwide***, each with multiple availability zones for redundancy.
Data Centers: Physical facilities hosting servers, networking, and storage.
Global Network: High-speed fiber backbone interconnecting regions for low latency.



2. Core Architectural Layers :

a. Compute Layer

Virtual Machines (VMs): On-demand compute instances.
Azure Kubernetes Service (AKS): Managed container orchestration.
App Services: PaaS for web/mobile apps.
Functions: Serverless compute for event-driven workloads.

b. Storage Layer

Blob Storage: Object storage for unstructured data.
Disk Storage: Persistent disks for VMs.
File Storage: SMB-based shared storage.
Queue/Table Storage: Messaging and NoSQL persistence.

c. Networking Layer

Virtual Networks (VNet): Isolated network environments.
Load Balancers: Distribute traffic across resources.
ExpressRoute: Private connections to Azure.
Content Delivery Network (CDN): Global caching for performance.

d. Identity & Access

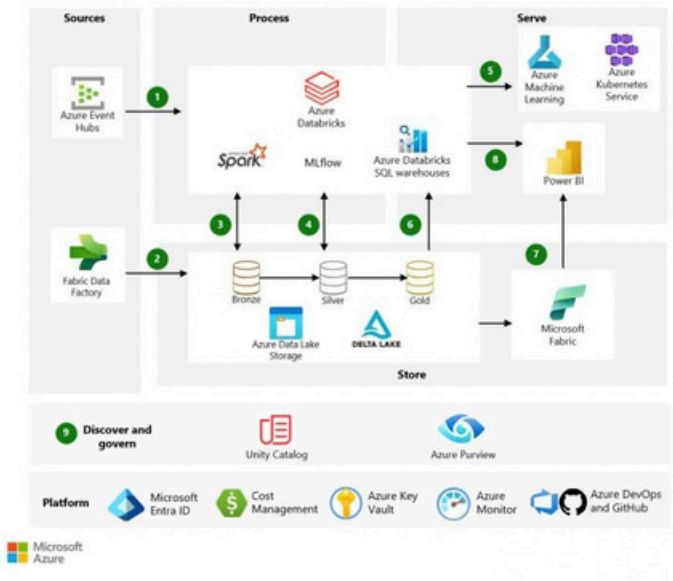
Azure Active Directory (AAD): Centralized identity management.
Role-Based Access Control (RBAC): Fine-grained permissions.
Microsoft Entra: Extended identity and governance.

3. Data& Analytics Layer :

Azure SQL Database: Managed relational database.
Cosmos DB: Globally distributed NoSQL database.
Synapse Analytics: Enterprise data warehousing.
Databricks: Big data and AI analytics.
Microsoft Fabric: Unified analytics platform.

4. AI &Machine Learning Layer :

Azure Machine Learning: Model training, deployment, and MLOps.
Azure Cognitive Services: Pre-built AI APIs (vision, speech, language).
Azure OpenAI Service: Generative AI integration.



5. Security & Compliance :

Azure Security Center: Unified security management.
Key Vault: Secrets and key management.
Defender for Cloud: Threat detection and protection.
Compliance: Meets global standards (ISO, GDPR, HIPAA).

6. Management & Governance :

Azure Resource Manager (ARM): Declarative resource deployment.
Azure Policy: Enforce compliance rules.
Cost Management: Monitor and optimize spending.
Monitoring & Logging: Azure Monitor, Log Analytics.

7. Well-Architected Framework :

Azure's design principles are organized into five pillars:

Reliability: Multi-region failover, availability zones.
Security: Defense-in-depth, encryption, RBAC.
Cost Optimization: Autoscaling, reserved instances.
Operational Excellence: Automation, DevOps integration.
Performance Efficiency: Elastic scaling, caching, CDN.

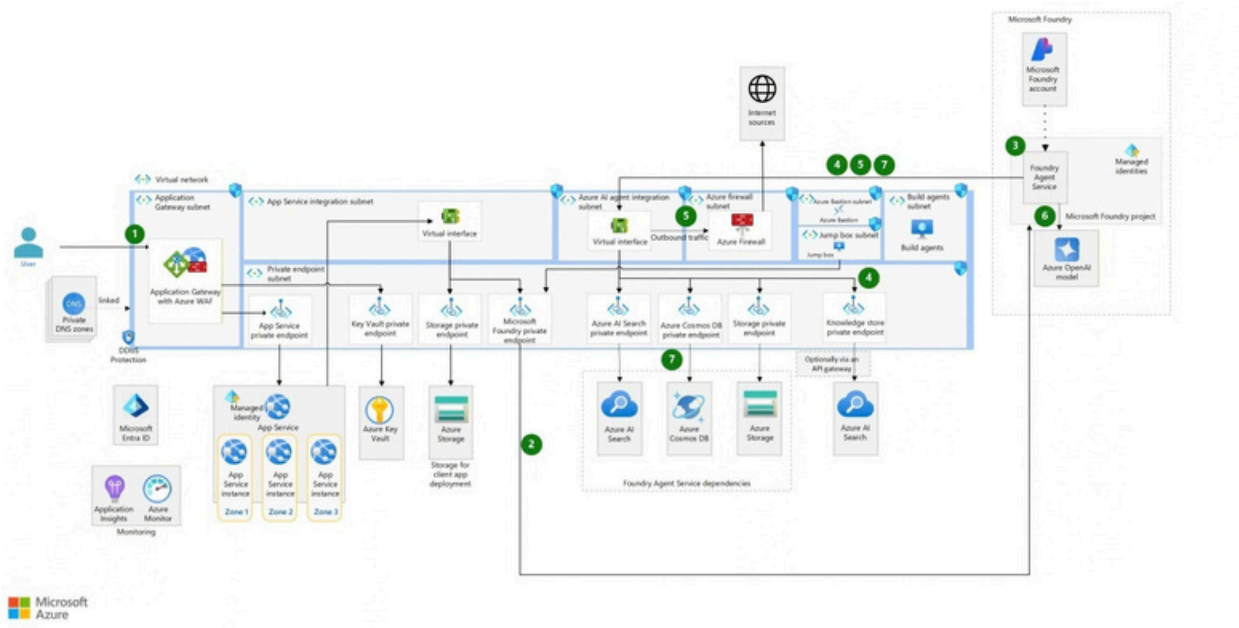
8. Hybrid & Edge Integration :

Azure Arc: Manage resources across on-premises, multi-cloud, and edge.
Azure Stack Hub/Edge: Extend Azure services to local environments.
IoT Hub: Connect and manage IoT devices.

9.Architecture Diagram :

Atypical Azure architecture includes:

Front-end: Web apps, APIs, mobile apps.
Middle tier: App Services, AKS, Functions.
Data tier: SQL, Cosmos DB, Blob Storage.
Integration: Service Bus, Event Grid.



10. Analysis

Strengths: Global reach, hybrid flexibility, integrated AI, strong compliance.
Challenges: Complexity of services, cost management, learning curve.
Use Cases: Enterprise apps, AI workloads, IoT, hybrid cloud, mission-critical systems.

